Изследване на конвейерни компютърни архитектури (Част 2)

Цел: Изследване и оптимизация на комуникиращите си процеси в конвейерни компютърни архитектури в условията на конвейерни състезания:

Теоретична част:

- Проблеми на конвейерната обработка – конвейерни състезания

Изпълнение на експерименталната част:

1. Да се проведе експериментално изследване на 5-фазен линеен конвейер с помощта на лабораторния симулатор.
   - Създайте нова програма и я наименувайте Pipeline_Ex2.
   - Въведете показания по-долу програмен фрагмент и го съхранете.
   - Компилирайте програмата и заредете кода в паметта за програми. Анализирайте генерираната последователност от процесорни инструкции, необходима за неговото изпълнение. Каква е очакваната стойност, която ще бъде установена в регистър R03 след завършване изпълнението на програмата?

```
MOV #1, R01
MOV #5, R03
MOV #3, R01
ADD R01, R03
HLT
```

   - Щракнете върху бутон SHOW PIPELINE и изследвайте представяната в прозорец „Instruction Pipeline“ информация.
   - Поставете отметка върху „Stay on Top“ (секция „Control“), за да задържите отворения прозорец най-отгоре. Уверете се, че има поставена отметка върху „No instruction pipeline“ и няма такава за „No history recording“ (секция „Control“).
   - Стартирайте програмата Наблюдавайте и анализирайте получаваните в прозорец „Instruction Pipeline“резултати. Снемете времедиаграмата и статистическите данни от изпълнение на инструкционния поток (секция „Statistics“). Каква е получената стойност в регистър R03? Полученият резултат отличава ли се от очаквания? Обосновете отговора си.
   - Премахнете отметката поставена върху „No instruction pipeline“ и поставете отметка върху „Do not insert bubbles“ (секция „Control“). Рестартирайте програмата и изпълнете отново. Наблюдавайте и анализирайте получаваните в прозорец „Instruction Pipeline“ резултати. Снемете времедиаграмата на изпълнение на инструкционния поток. Каква е стойността установена в регистър R03. Коментирайте различията с предходното изпълнение на програмата.
   - Вмъкнете инструкция NOP след третата инструкция, както е показано.

```
MOV #1, R01
MOV #5, R03
MOV #3, R01
NOP
ADD R01, R03
HLT
```

   - Рестартирайте програмата и я изпълнете отново. Каква е установената стойност в регистър R03?
   - Обобщете направените до тук наблюденията върху промените в съдържанието на регистър R03.
   - Премахнете инструкция NOP от горната програма и отметката „Do not insert bubbles“ (секция „Control“). Рестартирайте програмата и изпълнете отново. Каква е установената стойност в регистър R03? Снемете времедиаграмата от изпълнение на инструкционния поток и статистическите данни от изпълнение на инструкционния поток (секция „Statistics“). Съпоставете резултата с този от предходната стъпка като отбележите настъпилите промени?

- Обобщете направените до тук наблюдения като съпоставите получените резултати с тези от предходната стъпка на изпълнение.

2. Изследвайте ефективността на техника „*operand forwarding*“ за минимизиране влиянието на зависимостите по общи данни в информационния поток (data hazards).

   - Поставете отметка на „Enable operand forwarding“ (секция „Optimizations“, прозорец “Instruction Pipeline”) и изпълнете кодът от предишната задача отново.

   - Анализирайте времедиаграмата от изпълнение на инструкционния поток и статистическите данни от изпълнение на инструкционния поток (секция „Statistics“)? Съпоставете резултатите с тези от предишното изпълнение? Какви са настъпилите промени?

   - Обобщете направените наблюдения.

3. Изследване на оптимизационна техника „*Loop unrolling*“ за минимизиране на зависимостите по управление (control hazards). В основата на метода за оптимизация “*loop unrolling*” дублирането на кода на вътрешния цикъл толкова пъти, колкото е броя на итерациите на цикъла, премахвайки излишния код и включвайки инструкции за сравнение и безусловен преход (jump).

   - Създайте нова програма и я наименувайте Pipeline_Ex4_1.

   - Въведете показания по-долу програмен фрагмент и го съхранете.

```
program Pipeline_Ex4_1
  for n = 1 to 8
    t = t + 1
  next
end
```

   - Поставете отметки върху “Enable optimizer” и „Redundant Code“ (секция „Compiler“, раздел “Optimize”, прозорец „Program Compiler“).

   - Компилирайте програмата. Заредете кода на програма „Pipeline_Ex4_1“ в „CPU simulator’s memory“ и я изпълнете. Анализирайте генерираната последователност от процесорни инструкции, необходима за неговото изпълнение. Отбележете размера на генерирания за програма Pipeline_Ex4_1 код. Снемете времедиаграмата и статистическите данни от изпълнение на инструкционния поток (секция „Statistics“). Какви са получените данни за CPI, SF, No of instructions?

   - Уверете се, че има поставени отмети за „Loop Unrolling“ и „Redundant Code optimization“ (секция „Compiler“, раздел “Optimize”, прозорец „Program Compiler“). Преименувайте програмата на Pipeline_Ex4_2. Прекомпилирайте я, заредете я в „CPU simulator’s memory“ и я изпълнете. Анализирайте генерираната последователност от процесорни инструкции, необходима за неговото изпълнение. Отбележете размера на генерирания за програма Pipeline_Ex4_2 код. Снемете времедиаграмата и статистическите данни от изпълнение на инструкционния поток (секция „Statistics“). Какви са получените данни за CPI, SF, No of instructions?

   - Обобщете и коментирайте наблюдаваните резултати от изпълнението на програми Pipeline_Ex4_1 и Pipeline_Ex4_2.

4. Изследване на оптимизационна техника „*re-arranging instruction sequence*“ за минимизиране влиянието на зависимостите по общи данни в информационния поток (data hazards)

   - Уверете се, че има поставени отметки на „Show dependencies“ и „Redundant Code optimization“. Създайте нова програма. Наименувайте я Pipleine_Ex5_1 и я съхранете. Въведете дадения по-долу фрагмент от код, компилирайте го и го заредете в паметта.

```
program Pipleine_Ex5_1
  a = 1
  b = a
  c = 2
end
```

   - Компилирайте програмата и копирайте генерираната инструкционна последователност (без да копирате адресите на инструкциите).

   - Поставете отметка върху „Code Dependencies“ (секция “Compiler”, раздел “Options”, прозорец „Instruction Pipeline“). Преименувайте програмата на Pipleine_Ex5_2, компилирайте я и заредете в „CPU simulator’s memory“. Копирайте генерираната инструкционна последователност (без да копирате адресите на инструкциите).

- Съпоставете резултатите от изпълнението на двете инструкционни последователности. Коментирайте дали пренареждането на инструкциите, води до промяна логиката на програмата. Обосновете логиката на извършеното пренареждане.
- Изпълнете програми Pipleine_Ex5_1 и Pipleine_Ex5_2 и отбележете какви са получените стойности за CPI и SF?
- Внесете показаните промени в горната програма. Поставете отметка върху „Code Dependencies“ и съхранете програмата под име Pipleine_Ex5_3. Премахнете отметката и съхранете отново програмата под име Pipleine_Ex5_4.

```
program Pipleine_Ex5_1
  for n = 1 to 50
    a = 1
    b = a
    c = 2
  next
end
```

- Обобщете статистическите данни от изпълнение на инструкционния поток (секция „Statistics“). Какви са получените данни за CPI, SF, Data Hazards, Control Hazards? Коментирайте наблюдаваните промени след изпълнението на всяка от програмите. Наблюдава ли се подобрение и с колко процента?

5. Изследване на оптимизационна техника „Jump prediction“.
   - Създайте нова програма и я наименувайте Pipeline_Ex6
   - Въведете показания по-долу програмен фрагмент и го съхранете.

```
program Pipeline_Ex6
  i = 0
  for p = 1 to 40
    i = i + 1
    if i = 10 then
        i = 0
        r = i
    end if
  next
end
```

   - Изпълнете програмата и анализирайте статистическите данни от изпълнение на инструкционния поток (секция „Statistics“).
   - Поставете отметка върху „Enable jump prediction“ (секция „Optimizations“, прозорец “Instruction Pipeline”). Рестартирайте програмата и я изпълнете отново. Анализирайте повторно статистическите данни от изпълнение на инструкционния поток (секция „Statistics“). Обобщете и съпоставете получените от двете изпълнения резултати. Обосновете настъпилите промени.
   - Щракнете върху бутон „SHOW JUMP TABLE“. Анализирайте структурата на Jump Predict Table. Какви данни съхранява тази таблица?